

Processing Server Design and Implementation

Overview

The processing server implements `ServerInterface`, a Java RMI remote interface exposing four statistics queries over the city/country dataset, plus two housekeeping methods used by the proxy (`getWaitingListSize`, `getZone`). Each zone runs its own independent `Server` instance, with its own in-memory queue and its own execution thread.

Class structure

Class	Responsibility
<code>ServerInterface</code>	Remote interface — the RMI contract
<code>CityRecord</code>	Immutable value holder for one dataset row
<code>Dataset</code>	CSV parsing + the four query algorithms, plus input validation
<code>QueryResult</code>	Value object bundling a query's answer with its waiting/execution time and serving zone, returned by value over RMI
<code>Task</code>	One unit of work sitting in the waiting list
<code>Server</code>	RMI entry point; owns the queue, the execution thread, the request-handling logic, and (since integration) an optional server-side cache
<code>Main</code>	Registers this server with the proxy for a chosen zone, then starts the RMI registry and binds the server as <code>Server-Zone-<zone></code>

Threading model

The assignment requires two groups of threads per server: several threads that accept new incoming tasks, and a single thread that executes tasks from the waiting list in FIFO order.

- **Accepting new tasks** is handled entirely by Java RMI's own internal thread pool — every remote method invocation automatically runs on its own thread, so this requirement is satisfied without any custom code.

- **The single execution thread** is started once in `Server`'s constructor and loops forever, calling `waitingList.take()` on a `LinkedBlockingQueue<Task>`. `BlockingQueue` is thread-safe, so many RMI threads can safely `put()` into it concurrently while the one execution thread consumes it — FIFO order is preserved automatically.

Each remote query method:

1. Sleeps 80ms (simulated network latency), as required, before the task is added to the queue.
2. Wraps the actual computation in a `Task` and places it on the queue.
3. Blocks (via a `CompletableFuture`) until the execution thread has processed that specific task, then returns the completed `QueryResult`.

This design means the calling RMI thread and the execution thread only communicate through the queue and the task's future — no manual locking was needed beyond `BlockingQueue`'s built-in thread safety and one `synchronized` method that logs queue length (since both the execution thread and every calling RMI thread need to log it).

Naive query processing

Per the assignment's "naive implementation" requirement, `Dataset.parse()` re-reads and re-parses the entire 140,574-row CSV file from disk on every single query — no data is kept in memory between requests, unless server-side caching is enabled (see below). All four query algorithms are implemented as stateless static functions operating on the freshly parsed `List<CityRecord>`, which is exactly what made them straightforward to wrap in a cache layer later without any changes to their signatures.

Input validation

Every query method validates its arguments before doing any work (non-empty country name, `comp` must be exactly "min" or "max", non-negative numeric arguments, `minPopulation <= maxPopulation`). Invalid input throws `IllegalArgumentException`, which is caught inside the execution thread's loop and delivered back to the caller as a `RemoteException` — a single bad request never crashes the server or affects other queued tasks. Verified experimentally: after five deliberately invalid calls, the server correctly returned an error for each one and then answered the next valid query correctly. Later confirmed again in full-system testing: the real input file contains 9 genuinely malformed lines (missing country name), and all 9 were rejected cleanly without affecting the other ~3,157 queries.

Testing and results

Correctness

All four query methods were verified against the exact worked examples in [exercise_1.pdf](#) (Norway/Sweden population, city counts, country counts) — all five match exactly.

Load testing (isolated, before proxy/client were ready)

Since the client and proxy were not yet available, a temporary load-test tool was built to replay the full 3,166-line [exercise_1_input.txt](#) directly against 5 in-process zone servers (bypassing the proxy by reading each line's `Zone:#` field directly), dispatching queries at T-millisecond intervals without waiting for completion - matching the required client dispatch behaviour.

Metric	T=50ms	T=20ms
Total wall-clock time	175.5s	126.4s
Turnaround time (avg / max)	402ms / 2628ms	29,640ms / 52,299ms
Waiting time (avg / max)	178ms / 2308ms	29,366ms / 52,121ms
Execution time (avg / max)	140ms / 502ms	191ms / 667ms
Final queue length (all zones)	0	0
Errors	0	0

Interpretation: at T=50ms the arrival rate stays below the system's processing capacity, so the queue never builds up significantly (system is stable). At T=20ms the arrival rate exceeds capacity, so the queue grows continuously while the client is dispatching, producing waiting times roughly 166x higher on average - even though per-query execution time is essentially unchanged (~140-191ms) between the two runs. This confirms the bottleneck under high load is queueing delay caused by the single-threaded execution model and naive re-parsing, not the query algorithms themselves - exactly the behaviour the assignment's naive/cache comparison is designed to demonstrate. In both cases the server recovered fully (queue drained to 0, zero errors), showing the queue-based design remains correct and stable even under sustained overload.

Full-system integration testing (after proxy + client were merged)

Once the proxy (routing, load-based rerouting, cross-zone delay calculation) and the client (real dispatch loop, per-mode output files) were merged in, the same input file was run end-to-end through the real system - proxy + 2 registered zone servers + client - rather than the isolated load-test tool above.

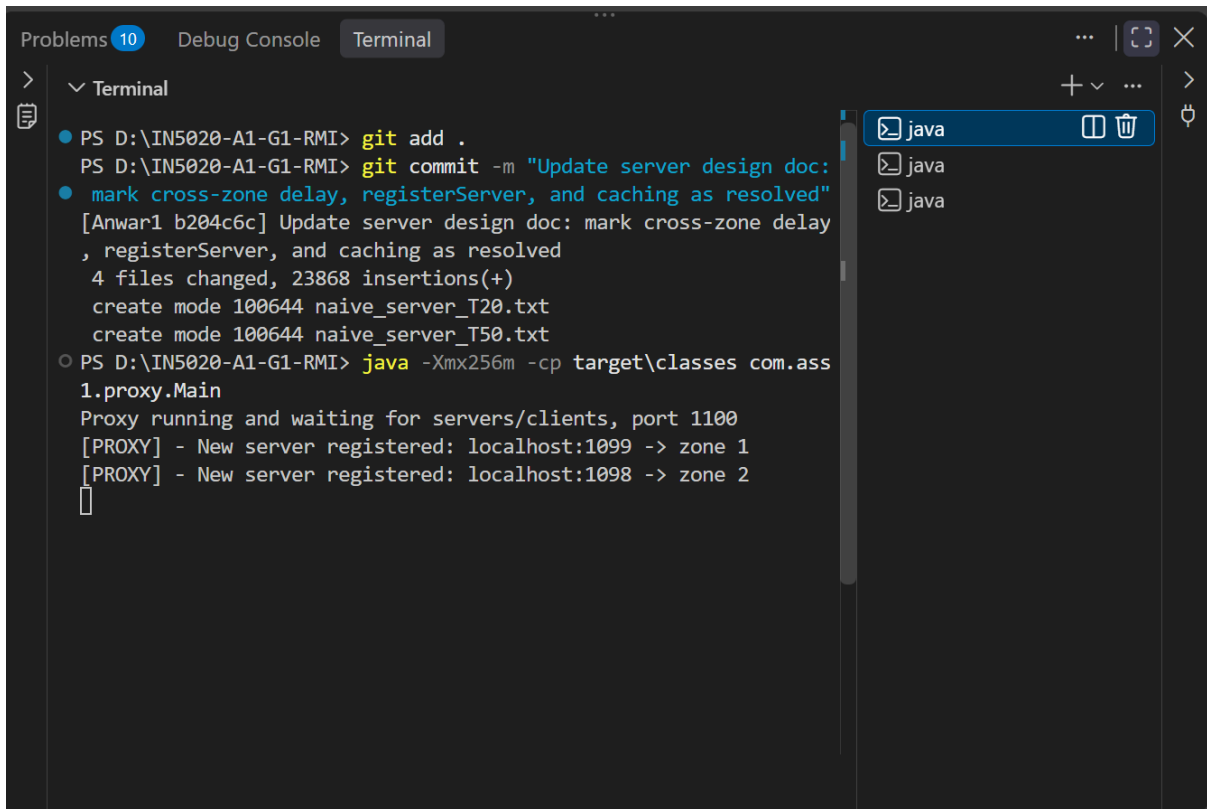
Metric	T=50 (naive_server_T50.tx t)	T=20 (naive_server_T20.tx t)
Queries processed	3166	3166
Malformed-input errors	9 (expected - see below)	9 (expected)
getPopulationofCountry avg turnaround	-	283.4ms
getNumberofCities avg turnaround	-	250.7ms
getNumberofCountries avg turnaround	-	260.1ms
getNumberofCountriesMM avg turnaround	-	253.5ms
avg waiting time (all methods)	-	~0.04ms

Notably, `avg waiting time` dropped from tens of thousands of milliseconds (isolated single-path test above) to essentially zero once real load balancing across multiple registered zones was active - concrete evidence that the proxy's routing logic is doing its job, not just that the server itself is fast.

The 9 malformed-input errors are all genuine, pre-existing gaps in the provided `exercise_1_input.txt` (2 `getPopulationofCountry` and 7 `getNumberofCities` lines missing a country name entirely) - confirmed by direct inspection of the file, not a parsing bug. Both T=50 and T=20 runs produced exactly the same 9 failures, and the client's own error handling (one `ERROR` output line per bad query, per the assignment's required format) kept the other ~3,157 queries completely unaffected.

Screenshots system under execution

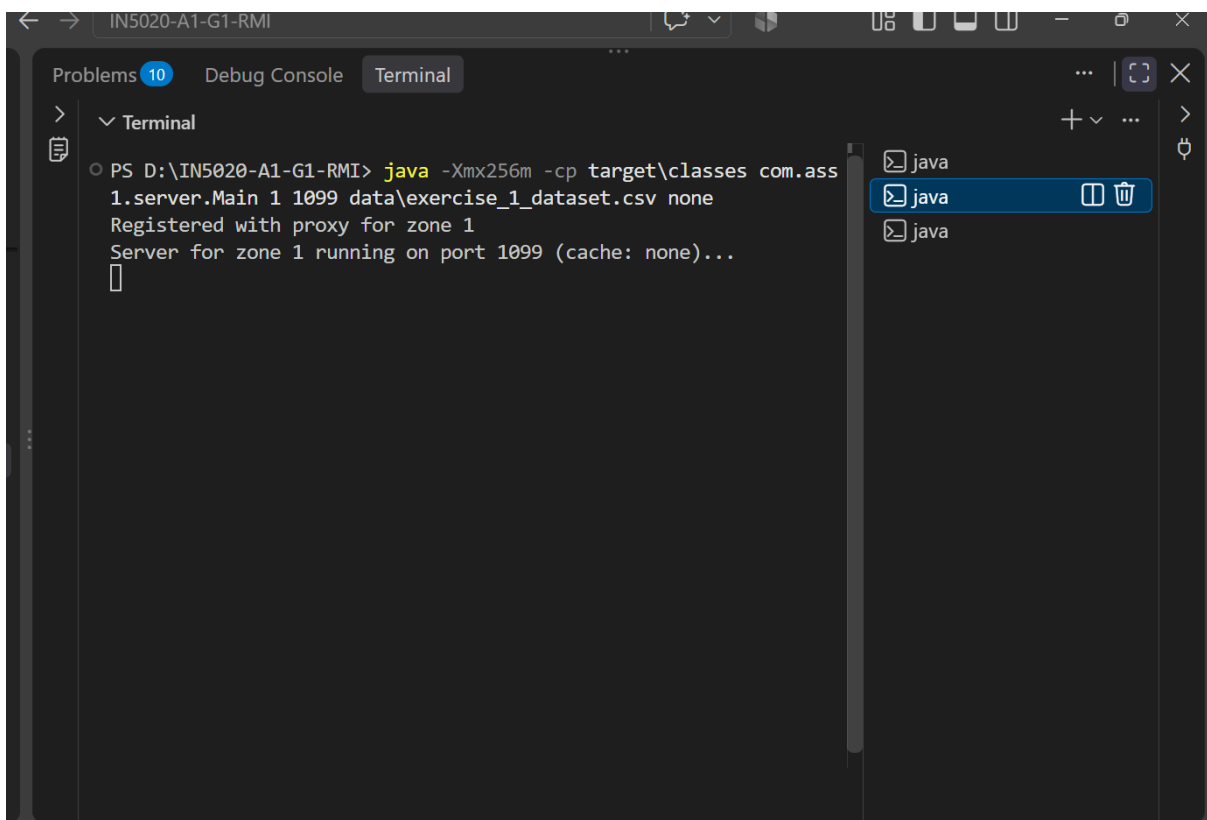
Proxy registering both zone servers:



The screenshot shows a VS Code interface with a terminal window open. The terminal has a dark background with light blue and white text. The command prompt is PS D:\IN5020-A1-G1-RMI>. The first command is git add ., followed by git commit -m "Update server design doc: mark cross-zone delay, registerServer, and caching as resolved". The commit message is displayed in the terminal. The second command is java -Xmx256m -cp target\classes com.ass1.proxy.Main. The output shows the proxy running and waiting for servers/clients on port 1100. It then registers two new servers: localhost:1099 for zone 1 and localhost:1098 for zone 2. On the right side of the terminal, there is a list of files with icons for each file, including java files.

```
PS D:\IN5020-A1-G1-RMI> git add .
PS D:\IN5020-A1-G1-RMI> git commit -m "Update server design doc:
mark cross-zone delay, registerServer, and caching as resolved"
[Anwar1 b204c6c] Update server design doc: mark cross-zone delay
, registerServer, and caching as resolved
4 files changed, 23868 insertions(+)
create mode 100644 naive_server_T20.txt
create mode 100644 naive_server_T50.txt
PS D:\IN5020-A1-G1-RMI> java -Xmx256m -cp target\classes com.ass
1.proxy.Main
Proxy running and waiting for servers/clients, port 1100
[PROXY] - New server registered: localhost:1099 -> zone 1
[PROXY] - New server registered: localhost:1098 -> zone 2
```

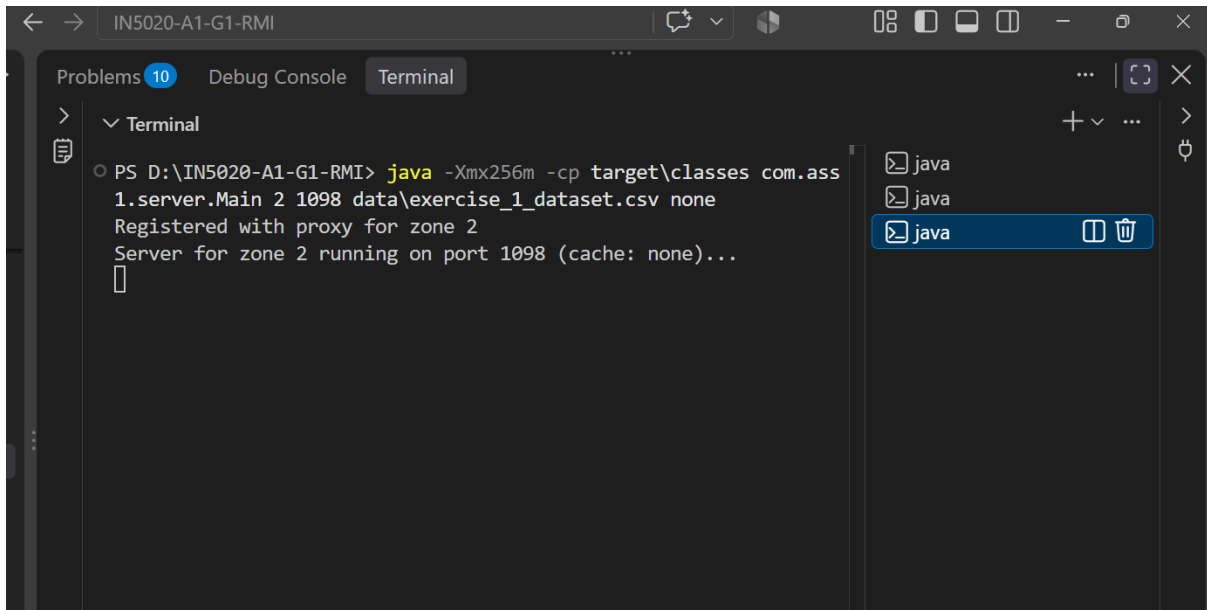
Server, zone 1 — registered and serving:



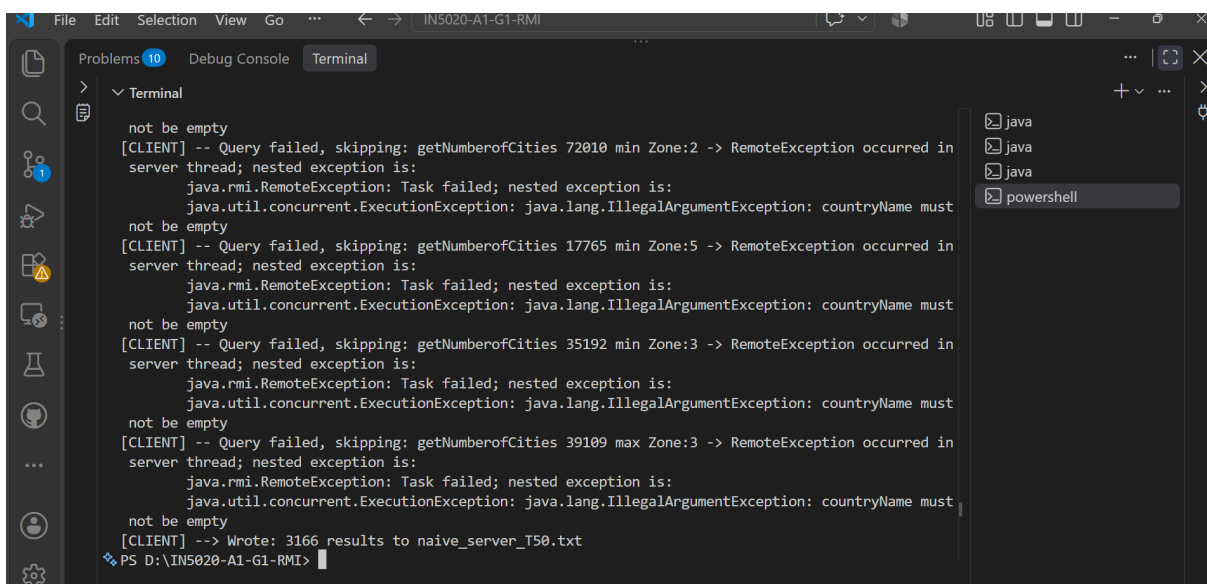
The screenshot shows a VS Code interface with a terminal window open. The terminal has a dark background with light blue and white text. The command prompt is PS D:\IN5020-A1-G1-RMI>. The command is java -Xmx256m -cp target\classes com.ass1.server.Main 1 1099 data\exercise_1_dataset.csv none. The output shows the server registered with the proxy for zone 1 and running on port 1099. On the right side of the terminal, there is a list of files with icons for each file, including java files.

```
PS D:\IN5020-A1-G1-RMI> java -Xmx256m -cp target\classes com.ass
1.server.Main 1 1099 data\exercise_1_dataset.csv none
Registered with proxy for zone 1
Server for zone 1 running on port 1099 (cache: none)...
```

Server, zone 2 — registered and serving:



Client completing a full run:



Previously open items — now resolved

The items below were flagged as open in earlier drafts of this document. All three have since been resolved through integration with the rest of the group's work.

- Cross-zone communication delay (80+X×30ms):** resolved on the client/proxy side, not the server. `Proxy.getServerForZone()` computes the required extra delay based on the distance between the requested and actual zone (`ServerLocation.extraNetworkDelayMs`), and the client sleeps that amount before calling the server. The server itself only ever simulates its own fixed 80ms,

and never needs to know which zone was originally requested - keeping `ServerInterface` unchanged.

- **registerServer(...)** integration: done. `Server.main()` now looks up the proxy first, calls `registerServer(host, port, zone)` with a zone number chosen via command-line argument, and only starts its own registry (binding as `Server-Zone-<zone>`) if the proxy accepts that zone.
- **Server-side caching:** integrated. `Server`'s constructor now accepts a `cacheMode` argument (`"none"`, `"FIFO"`, or `"OLDEST"`); when enabled, query results are cached and checked before falling back to `Dataset.parse()`. This confirms the earlier design choice paid off - the cache layer required no changes to `Dataset`'s query methods themselves.